Tanaka Certificates: Weekly Research Report

Filip Morawiec

6 July 2026

Abstract

This week I contributed by implementing a multidimensional verifier for ReLU neural networks (with last layer being piecewise quadratic to ensure completeness) and proving it's correctness [TODO]. After meeting with Greg, he recommended that I should focus on proving the correctness of the ReLU NN region discovery algorithm and algebraically working through an example 2D SDE verification, which I did and present here. I also worked on training a certificate for this problem - following the techniques used by Greg in his previous paper - but I failed to satisfy all the constraints a certificate should have.

1 Multidimensional verifier

1.1 Problem setup

Consider a multi-dimensional stochastic differential equation

$$dX_t = f(X_t)\,dt + g(X_t)\,dW_t \tag{1}$$

on a domain $D \subset \mathbb{R}^d$. For a reach–avoid problem, let $X_0 \subseteq D$ be the initial set, $X_u \subseteq D$ the unsafe set, and $X_\tau \subseteq D$ the target. We seek a certificate V and constants $\alpha < \beta$ and $\varepsilon > 0$ such that

1. boundary condition holds:
$$\sup_{x \in X_0} V(x) \le \alpha \qquad \text{and} \qquad \inf_{x \in X_u} V(x) \ge \beta \tag{2}$$
2. generator condition holds:
$$\mathcal{L}V(x) \le -\varepsilon \qquad \text{for } x \in \{V \le \beta\} \setminus X_\tau, \tag{3}$$

where, at smooth points,

$$\mathcal{L}V(x) = V'(x)f(x) + \frac{1}{2}V''(x)g(x)^2. \tag{4}$$

The process is stopped after reaching the target or unsafe set: once the reach–avoid property has been achieved or violated, its later trajectory is irrelevant.

Remark. We model a certificate V as:

$$V(x) = \sum_{k=1}^{m} \lambda_k \phi(z_k(x)) + c, \tag{5}$$

$$z(x) = \texttt{NN}(x) = W_L\texttt{ReLU}(\ldots\texttt{ReLU}(W_1 x + b_1)\ldots) + b_L, \tag{6}$$

$$\phi = \texttt{PiecewiseQuadratic1D}. \tag{7}$$

This way, because we include a piecewise quadratic function ϕ in the last layer, we cover a certain failure case related to the curvature term $V''(x)$ in the generator condition (3).

2 Correctness proof of 1D ReLU NN region discovery

Consider the following Algorithm 1 which discovers the linear cells of a one-dimensional ReLU network.

Listing 1: Discover one-dimensional linear cells of a ReLU network

```python
def discover_cells(layers):
    """
    The network has a one-dimensional input and a one-dimensional output, but hidden
    layers can be multi-dimensional:
    >>> layers[i] = (W_i, b_i)
    >>> W_i.shape = (n_{i+1}, n_i)
    >>> b_i.shape = (n_{i+1},)
    """

    # A cell is (lower, upper, slope, intercept).
    cells = [(-inf, inf, [1], [0])]

    for W, b in layers:
        new_cells = []
        for lower, upper, slope, intercept in cells:
            # key observation: in a cell C_prev(l, u, slope, intercept), we have
            #   z_prev = slope @ x + intercept, so the linear function
            # in the current cell C(?, ?, alpha, beta) will be
            #   z = W @ z_prev + b = (W @ slope) @ x + (W @ intercept + b)
            alpha = W @ slope
            beta = W @ intercept + b

            roots = {
                -beta[j] / alpha[j]
                for j in range(len(beta))
                if alpha[j] != 0
                and lower < -beta[j] / alpha[j] < upper
            }
            boundaries = [lower, *sorted(roots), upper]

            for left, right in consecutive_pairs(boundaries):
                x = choose_point(left, right)
                active = alpha @ x + beta > 0 # list of booleans
                new_slope = where(active, alpha, 0)
                new_intercept = where(active, beta, 0)
                new_cells.append(
                    (left, right, new_slope, new_intercept)
                )

        cells = new_cells

    return cells
```

It discovers the linear cells of a one-dimensional ReLU network, with:

$$\texttt{NN}(x) = W_L\texttt{ReLU}(\dots\texttt{ReLU}(W_1x + b_1)\dots) + b_L. \tag{8}$$

To prove the correctness algorithms, the key step is to show, given a linear cell of the network up to layer $m-1$, how to discover the linear cells of the network up to layer m.

As a precondition, we already have a linear cell of the network when we begin: `C(-∞, ∞, [1], [0])` represents the entire real line, with slope [1] and intercept [0].

Now, assume at layer $m-1$ we have a linear cell `C(l, u, slope, intercept)` which is correctly discovered. Then, at layer m, as shown in the code comment, we have a linear function:

$$z = W\texttt{@}\ z_{m-1} + b = (W\texttt{@}\ \text{slope})\texttt{@}x + (W\texttt{@}\ \text{intercept} + b) \tag{9}$$

Since z is a list of n_m elements, we have to check when $z_j = 0$ for each $j = 1, \dots, n_m$. This gives us the new potential breakpoints.

Now, to determine how the resulting cells are formed, we can take a point within each interval $(\texttt{breakpoint}_i, \texttt{breakpoint}_{i+1})$ and check whether the resulting neuron would be active. This yields the new slope and intercept for that cell.

Note that in the actual implementation, the last layer is piecewise quadratic as noted in Remark 1.1, but the proof of correctness is similar

3 2D SDE verification example

4 Certificate training